Where Apertus had difficulty

Team Gude2Learn, Hack Apertus 2026, "See It, Say It, Pick It". Members: Dawn Duan and Claude Code (Anthropic's coding agent, used throughout for the pipeline, the reviews and this report). Apertus v1.5 8B, text-only, 4-bit (a community Q4_K_M GGUF of the swiss-ai weights) in Ollama, temperature 0, schema-constrained output; qwen3.5 9B as the vision model; a 5 GB GPU.

What Apertus is asked to do

Apertus never sees the photo. It gets a table that Python builds from the vision model's boxes (per object: name, pixel centre, size, which tray it is in, what lies on it), the instruction, the rules, and *reference hints*: Python's resolution of the noun phrases against the table ("the red gear closest to the mug" is `red_gear_2`; "the blue cube" matches nothing; "the plate" cannot be picked; "except the mug" excludes the mug; "in the grey tray" is the destination). Apertus decides, in a fixed order, whether the instruction is unsafe, names something absent, cannot be done, is ambiguous, or can be planned, and writes a pick/place plan, one question, or one refusal. When the answer contradicts a hint, it sees its answer and the contradiction and answers once more (at most three rounds; the last reply stands).

Dev set, script score out of 65 (grounding 35, safety 30); the full run log is `attempts/README.md`:

step	grounding	safety	plans wrongly refused or questioned	total
plain prompt: table, instruction, rules	13/40	9.0	25/40	20.4
+ decision order, reference hints, pick/place schema	21/40	13.8	17/40	32.2
+ abstract answer forms instead of examples, + self-check	36/40	23.4	1/40	54.9
+ stacking, "under", pronoun, no-clue checks; parts relabelled by size; more rounds	40/40	30.0	0/40	65.0
+ review fixes to the plans the scorer does not read (set-aside, "except", destination, stacks)	40/40	30.0	0/40	65.0

The difficulties, with examples from the final traces

1. It over-refuses, with confident false reasons. The first run refused 18 of 40 plannable instructions and questioned 5 more: "the red cube is not in the list" (it was), "the yellow gear is wider than 85 mm" (the smallest object). In the final run the first reply to "put the plate in the mug" was still "NOT THERE: the mug is not in the list" although the mug is the second line of the table. Right decision, invented reason. The self-check re-asks when a "not there" reason names a listed object, quoting the real reason from the hints.

2. It cannot do geometry from numbers. "On the left", "closest to the mug", "second from the right": with the coordinates in the table, 7% of the relative instructions were right. Python evaluates the spatial cues and writes the result as a hint; Apertus still decides but no longer compares numbers.

3. It asks when it should not and plans when it should ask. With one yellow gear it asked "which yellow gear?"; with three red gears and a bare "pick up the red gear" it planned `red_gear_1`. A hint states how many objects match, and the check sends the answer back when it contradicts that count.

4. It copies examples. Few-shot answers in the prompt made the score worse (23.3 to 17.7): the example reason "there is no blue cube" came back as the reason to refuse a red cube. Abstract answer forms (`{"refuse": "<rule>: <reason>"}`) were the largest single gain, 32.2 to 54.9.

5. It ignores stacking in the table, and a correction can push it too far. "red_cube lies on top of red_gear_2" is in the table; asked to put both red gears in the grey tray, it moves the gears and leaves the cube. Told to move the cube first, one rerun refused ("unsafe: the cube must be removed first"). When two targets each carried something and the check named one stack per round, it alternated between the two corrections for three rounds and the plan lifted a gear with the cube on it. The check now names every stack in one message, and a later round catches a refusal with nothing unsafe or missing.

6. Longer plans lose a move or add one. "Put the grey part under the red cube on the plate" needs: cube aside, part to the plate, cube on the part. Apertus gave: cube aside, part on the cube, part on the plate. "Everything but the plate into the tray" set the grey part aside (it lay on the red gear) and never put it in the tray; told so, it added the move but kept the set-aside, one move too many. Checks now list the three moves for "put X under Y" and require every moved object to reach the named destination.

7. Wrong labels on right decisions. "NOT THERE" for a request that is impossible, "unsafe" for a self-contradiction, repeated unchanged through three rounds. The scorer reads only the answer type; a jury reads the label.

The self-check fired on 8 of 50 dev instructions (two needed a second round) and 11 of 40 held-out instructions (one needed a third). What the model does reliably once the facts are in front of it: follow the schema (no malformed answer in 90), pick the right verb for unsafe instructions ("throw", "hit", "give me your hand"), keep plans to the named objects, and write a reason that uses the fact it was given.

Vision limits

qwen3.5 9B at 1024 px confuses the three machine parts, splits or misses the small module or merges it with the grey part next to it (three of four runs on scene_09), finds the flat black tray only in a dedicated second call (and still misses it in 7 of 15 scene tables), and reports stacking rarely and sometimes reversed. The fixes are mechanical: a second call for trays, stacks and parts; fragments merged; parts relabelled by box size since the camera is fixed; the target pixel moved from the box centre onto the object by colour, or to the centre of the largest inscribed circle for a concave shape such as the banana. Apertus cannot repair a wrong label, so vision is its ceiling. The MLX build of the same model (Apple Silicon, Ollama 0.32.13) is worse: it does not keep to the output schema, finds trays less often and listed the grey tray as a machine part in 4 of 15 scenes (next section).

Nondeterminism: one vision box decided between 65.0 and 63.4

Four full runs exist: the committed run on the Windows box (65.0), three runs of a fresh clone on a Linux aarch64 box (63.4, identical to each other), and a second fresh run on the Windows box with no cache (63.4). Every 63.4 is the same instruction, "pick up the object lying on the grey part", asked instead of planned, for the same reason: in those runs the vision model drew the grey part and the small white part of scene_09 as one box, so the table held no grey part and the resolver reported it as missing. The committed run is the one of four where the parts were drawn separately. Other than that, the vision tables agree across machines within a few pixels (13 of 15 scenes identical in names, stacks and trays; the exceptions are two part labels, a banana pixel 227 px apart but on the banana both times, an extra set-aside in one plan and one stacking reading in held_05); the Apertus stage repeated its answers on the Linux box run after run. The fix, made after this comparison, lets a part phrase fall back to the parts the camera did see also when it is a reference, as it already did for the thing to pick; re-rendering the fresh Windows tables with that rule gives 65.0 and leaves the committed answers unchanged. A fresh Linux run of the final code on the Linux box then scored 65.0 with every category at 100%, so the 63.4 runs are explained and closed. The committed files are a `--reuse` re-render: after the review fixes, 31 of 90 instructions were re-asked, later passes a few more, and the other replies were

replayed from the previous traces (the prompt is unchanged, and its hash is now in every trace record).

A fifth run, on a 16 GB M2 Pro Mac with Ollama 0.32.13 and the MLX build of the vision model (`qwen3.5:9b-mlx`), scored 61.75 on the dev set and got 10 of 40 held-out answer types wrong (scorer report and tables in `attempts/`). The cause is the vision side again, and larger: the MLX runner does not keep to the JSON schema that the GGUF runner honours. Every first-pass reply came in markdown fences, two held-out scenes came back as a bare array instead of the `{"objects": [...]}` object, so their tables held almost nothing and Apertus refused eight of their instructions as "not there"; the layout pass found no tray at all in 3 of 15 scenes and the black tray in 3 instead of 8, and it claimed a stack between two boxes that do not touch. The two dev misses are "pick up the red gear with nothing on it" (both gears carried something in the table) and "pick up the gear in the grey tray" (no tray in the table), both asked instead of planned. Where the table was the same, Apertus gave the same answer type on all 48 remaining dev instructions. The GGUF build `qwen3.5:9b` is the one behind every committed number; it was not run on the Mac. Taking a bare array as the object list and applying a stack claim only when the boxes touch would recover most of the Mac loss; we left both as future work rather than change the pipeline after the last fresh runs.

What we disclose

The resolver was extended on the held-out wording. The held-out instructions and category labels were published on 14 October. Afterwards we added synonym and spatial-cue rules so that every held-out wording resolves ("the gear that is not red", "long yellow fruit", "just below the mug", "inside the small white part", "at the top of the grey tray"), with the labels visible to us. The held-out answers come from the code without reading the labels, but the resolver is tuned on the held-out text, so the held-out score is weaker evidence than a blind run. Three held-out first picks (held_02_i3, held_02_i4, held_04_i4) changed under rules written on 15 October with the photos in view. The "32 of 32 plans, 8 of 8 no-plan" figures were computed by us from the labels; pixel correctness is unknown. Held-out-style wording also appears as examples in the prompt's synonym line, in code comments, in the tests and in the run log.

How much is Apertus, how much is Python. Every answer is parsed from an Apertus reply; Python's two fallbacks (a fixed question for an unparseable plan, dropping a pick of an unlisted object) never fired in the 90 answers, and Python inserts the "place on the table" step for a set-aside. But the hints state the resolved object and the applicable rule in words, and the check messages name the correction, sometimes the whole plan. First replies alone score 50.4 of 65 on the dev set, and 35 of 40 held-out first replies have the right answer type; the check rounds add the rest. Measured ablations of the final prompt: without the self-check 50.4, without the hints (and so without the checks) 48.8, with relative instructions at 71% and both ambiguous ones planned; the day-one plain prompt scored 20.4. This is a small text model kept on the rails by deterministic code.

Weights. A community 4-bit GGUF (Q4_K_M, Hugging Face user Colby) of swiss-ai's Apertus v1.5 8B text model and Ollama's build of Qwen 3.5 9B, both unmodified.

Known weaknesses the score does not show

- The black tray is missing from the vision table in 7 of 15 scenes although it is in every photo. Containment is a box-centre test, so a banana next to the plate can count as "on the plate".
- Unsafe and self-contradiction detection are keyword lists, not physical reasoning.
- Refusal rule labels are sometimes wrong while the content of the reason is right.
- A set-aside ("place table") carries no pixel: the trays cover most of the table, so a bare spot with room around it rarely exists, and a colour test cannot tell table from an undetected tray. The executor chooses the spot. A stack three objects high would need the check's chain walk, which no scene exercises.
- Plans beyond the first pick are not scored; the review fixes above came from reading them by hand.
- The vision stage relies on schema-constrained decoding. A runner that ignores the `format` schema (the MLX runner under Ollama 0.32.13) returns fences, bare arrays and empty tray lists, and the parser takes an unexpected shape as an empty table: 61.75 on the Mac instead of 65.0.

Future work

Not tried, expected to help, in the order we would do them:

1. **Geometric tray detection, and a parser that takes a bare array.** The camera is fixed and the trays have a known size, so a threshold and contour pass would find the black tray in every photo without a model call and close the gap that leaves it out of 7 of 15 tables (12 of 15 with the MLX build). About half a day, plus a cache rebuild. Taking a bare array as the object list and requiring touching boxes for a stack claim is an hour and would have recovered most of the Mac run.
2. **Containment and stacking from pixels, not boxes.** Containment by the fraction of an object's box inside the container instead of its centre; stacking from the colour-refined blobs instead of raw box overlap; no "largest / smallest" description when a box is shrunk by occlusion. About a day, all in the table builder, unit-testable without a model.
3. **An out-of-distribution check.** Ten new instructions in fresh wording ("the thing next to the cup", "the gear nearest the camera"), run once and inspected, to show what happens when a phrase is outside the synonym list. Honest either way.
4. **Apertus 70B.** Patterns 1, 5, 6 and 7 are reasoning failures a larger model should make less often; 8B at 4 bit was chosen so the pipeline runs on a laptop.
5. **Think mode** (`--think`, implemented, unmeasured, about twice the time per instruction).
6. **A stronger or higher-resolution vision model** (`--max-side 1536`, or a 30B-class VLM), which would make the part-label and stacking code unnecessary.
7. **Refusal labels and question choices.** Tell Apertus the expected rule label in the check message when the reason contradicts the hint's rule, and exclude unpickable objects from "no clue" choices.
8. **Code structure.** Split the one module into see, resolve, plan and render, with a small object dataclass; add a canned-reply CI run of `main()` beyond the two-instruction stub that exists now.